

Scénarios de test — Hublot

Document **opérationnel** pour **élaborer et exécuter** des scénarios de test, en complément du [PLAN_TEST.md](../1.3 Rédiger des scripts dans la démarche DevOps/1.3.7 Automatiser les tests en DevOps.md) et des [enjeux](../1.2 Préparer le déploiement d'une application/1.2.1 Les enjeux des plans de test.md).

Annexe livrable : [Annexe 13](#)

1. Ce qu'est un scénario de test

Un **scénario** décrit un comportement attendu de l'application dans un **cas d'usage identifié**. Il va plus loin qu'une ligne dans le plan de test :

Élément	Rôle
Préconditions	État du système avant l'action (données, session, URL)
Actions	Étapes ordonnées, reproductibles
Résultats attendus	Critères observables (OK / non OK)
Trace	Identifiant stable + lien vers ligne du plan

Convention de rédaction : chaque scénario utilise la structure **Étant donné / Quand / Alors** (équivalent Gherkin Given / When / Then), largement utilisée en test et en Agile.

2. Correspondance avec le plan de test

ID scénario	Ligne du tableau [PLAN_TEST.md](../1.3 Rédiger des scripts dans la démarche DevOps/1.3.7 Automatiser les tests en DevOps.md)
ST-F01	Test authentification
ST-F02	Test chargement des données

ID scénario	Ligne du tableau [PLAN_TEST.md](../1.3 Rédiger des scripts dans la démarche DevOps/1.3.7 Automatiser les tests en DevOps.md)
ST-F03	Test des filtres
ST-F04	Test responsive
ST-F05	Test performance
ST-F06	Test badge environnement
ST-SEC01	Test sécurité (headers)
ST-SEC02	Garde-fous applicatifs (sanitization, validation, session)
ST-SEC03	Aucun secret commité (scan Gitleaks)
ST-SEC04	Dépendances de production maîtrisées (audit + Trivy)
ST-SEC05	Analyse statique du code (SAST CodeQL)
ST-AUTO-*	Lint, tests unitaires, E2E, build, audit

Outils & stratégies détaillés :

[OUTILS_STRATEGIES_TESTS_SECURITE.md](../1.2 Préparer le déploiement d'une application/1.2.4 Les outils et les stratégies des tests de sécurité.md).

3. Méthode d'élaboration (checklist Studi)

1. **Identifier l'acteur** : utilisateur interne RH, développeur, CI.
2. **Définir le périmètre** : quel écran, quelles données métier réelles uniquement après connexion habilitée.
3. **Formuler une intention** en une phrase (ex. « un utilisateur non authentifié ne voit pas le tableau de bord »).
4. **Découper en étapes atomiques** (une action par étape lorsque possible).
5. **Rendre observable le succès** : texte visible, code HTTP, absence d'erreur console.

6. **Choisir l'environnement** : préférer **STAGING** pour les tests manuels hors DEV — voir [ENVIRONNEMENT_TEST.md](../1.1 Les bases de la démarche DevOps/1.1.3 Les bases d'un environnement de test.md).
-

4. Scénarios fonctionnels (manuels)

ST-F01 — Connexion refusée sans identifiants valides

Champ	Détail
Priorité	Critique
Type	Manuel — sécurité / accès
Plan	Test authentification
Environnement conseillé	STAGING puis PROD (à valider après STAGING)
Préconditions	Aucune session active (nouvelle fenêtre privée ou <code>sessionStorage</code> vidé). URL de déploiement connue (variables Netlify définies).

Étant donné que l'utilisateur ouvre la page d'accueil de Hublot **sans être connecté**,

Quand il saisit un identifiant ou un mot de passe **incorrect** et valide le formulaire,

Alors un message d'erreur explicite s'affiche **et** le tableau de bord (onglets, graphiques) **n'est pas** accessible.

Étant donné que l'utilisateur saisit les **identifiants valides** (conformément aux variables d'environnement du déploiement),

Quand il soumet le formulaire,

Alors le tableau de bord s'affiche **et** le bouton permettant de se déconnecter est visible (`title="Se déconnecter"`).

Étant donné que l'utilisateur est connecté,

Quand il déclenche la déconnexion et confirme si une boîte de dialogue apparaît,

Alors il revient à l'écran de connexion sans accès résiduel au contenu métier tant qu'il ne se reconnecte pas.

ST-F02 — Chargement des données sur les principaux écrans

Champ	Détail
Priorité	Critique
Type	Manuel — données / intégration
Plan	Test chargement des données
Préconditions	Utilisateur connecté . Données réelles configurées selon la procédure interne (<code>agents.json</code> / Neon — hors dépôt public).

Étant donné que l'utilisateur est authentifié,

Quand il ouvre successivement au minimum trois onglets distincts (par ex. **Vue d'ensemble**, **Vue dynamique**, **effectifs** ou **missions**),

Alors chaque écran affiche des **éléments valorisés** (chiffres, graphiques ou tableaux non vides lorsque les données métier sont présentes),

Et aucune erreur **bloquante** n'apparaît dans la console développeur (F12 → Console) pour ces navigations,

Et les libellés de métier restent lisibles et cohérents avec la DIRM Méditerranée (pas de page blanche).

ST-F03 — Filtres globaux et cohérence des vues

Champ	Détail
Priorité	Haute
Type	Manuel — logique métier
Plan	Test des filtres
Préconditions	

Champ	Détail
	Utilisateur connecté sur un onglet avec filtres actifs (barre « Filtres »).

Étant donné que les filtres sont à leur valeur par défaut (« tous » ou équivalent),

Quand l'utilisateur sélectionne une **région** ou un **service** dans la liste (ex. filtre « DIRM Méditerranée » si présent),

Alors les indicateurs et graphiques de l'onglet courant se **mettent à jour** sans rechargement complet anormal,

Et la sélection reste visible tant qu'il ne réinitialise pas les filtres.

Quand l'utilisateur réinitialise les filtres (bouton ou action prévue),

Alors la vue revient à l'état agrégé initial.

ST-F04 — Utilisation sur petit écran (responsive)

Champ	Détail
Priorité	Moyenne
Type	Manuel — ergonomie
Plan	Test responsive
Préconditions	Navigateur Chromium ou équivalent avec outils développeur.

Étant donné une largeur viewport **375 px** (mobile type),

Quand l'utilisateur fait défiler la page et utilise les **onglets** et la zone **filtres**,

Alors le contenu ne déborde pas horizontalement de manière gênante,

Et au moins un graphique principal reste lisible ou accessible par défilement vertical.

ST-F05 — Réactivité perçue (performance)

Champ	Détail
Priorité	Moyenne
Type	Manuel — perception
Plan	Test performance
Préconditions	Connexion possible sur PROD ou STAGING.

Étant donné que la page d'accueil métier est chargée (connexion OK),

Quand l'utilisateur enchaîne **trois changements d'onglet** en moins de 15 secondes,

Alors chaque transition s'effectue avec un délai **acceptable** (< 5 s perçues en réseau standard, hors cas extrême),

Et aucun gel prolongé sans retour utilisateur.

ST-F06 — Distinction environnement développement vs production

Champ	Détail
Priorité	Basse
Type	Manuel — environnement
Plan	Test badge environnement
Préconditions	Accès DEV local (<code>npm run dev</code>) et navigateur pour PROD.

Étant donné l'application en **mode développement** local selon la doc,

Quand l'utilisateur affiche login ou tableau de bord,

Alors le **badge d'environnement** (étiquette type « Développement ») est visible conformément au code.

Étant donné l'URL de **production** officielle (`dirmhublot.netlify.app`),

Quand une session valide ou l'écran de login s'affiche,

Alors aucun badge orange / indication « staging » ou « preview » prévu pour les non-productions n'est affiché (comportement actuel).

5. Scénarios sécurité

ST-SEC01 — Headers HTTP en production

Champ	Détail
Priorité	Haute
Type	Semi-automatisable (CLI)
Plan	Test sécurité (headers)
Trace	[SECURITE_4_DEPLOIEMENT.md](./1.2 Préparer le déploiement d'une application/1.2.4.2 Sécurité du déploiement.md), Annexe 02

Étant donné le site déployé en HTTPS sur Netlify,

Quand l'exécutant lance :

```
curl -sI https://dirmhublot.netlify.app
```

Alors la réponse utilise **HTTPS** ou une redirection équivalente sûre,

Et au moins les en-têtes **X-Frame-Options** et **X-Content-Type-Options** sont présents avec des valeurs restrictives (**Annexe 02**),

Et le statut HTTP est acceptable (200, 304, etc.).

Automatisation : `npm run security:headers` (script `scripts/check-security-headers.sh`) — sortie 0 si les en-têtes obligatoires sont présents.

ST-SEC02 — Garde-fous applicatifs (sanitization, validation, session)

Champ	Détail
Priorité	Haute
Type	Automatisé (Vitest)
Plan	Test sécurité (code)
Trace	<code>src/utils/security.test.ts</code> , [OUTILS_STRATEGIES_TESTS_SECURITE.md](../1.2 Préparer le déploiement d'une application/1.2.4 Les outils et les stratégies des tests de sécurité.md)

Étant donné les utilitaires de sécurité (`sanitizeInput` , `validateFilter` , masquage RH, session),

Quand la suite `npm run security:test` est exécutée,

Alors une entrée contenant des balises/ `javascript: /handlers` est nettoyée, un filtre hors liste est refusé, les données RH sont masquées, et une session de plus de 8 h est expirée.

ST-SEC03 — Aucun secret commité (Gitleaks)

Champ	Détail
Priorité	Critique
Type	Automatisé (CI — bloquant)
Plan	Test sécurité (secrets)
Trace	<code>.github/workflows/security-scan.yml</code> , <code>.gitleaks.toml</code>

Étant donné l'arborescence du dépôt,

Quand Gitleaks scanne les fichiers (hors faux positifs allowlistés : identifiants de démo/test),

Alors aucun secret réel n'est détecté (le job échoue sinon).

ST-SEC04 — Dépendances de production maîtrisées

Champ	Détail
Priorité	Critique
Type	Automatisé (CI — audit bloquant, Trivy informatif)
Plan	Test sécurité (dépendances)
Trace	<code>ci.yml</code> (job audit), <code>security-scan.yml</code> , [SECURITE_AUDIT.md](./1.2 Préparer le déploiement d'une application/1.2.4.3 Audit des dépendances.md)

Étant donné les dépendances de production,

Quand `npm run audit:prod` puis Trivy s'exécutent,

Alors aucune CVE critical (ni high hors allowlist) n'est présente ; Trivy remonte le reste pour revue.

ST-SEC05 — Analyse statique du code (SAST)

Champ	Détail
Priorité	Haute
Type	Automatisé (CI — informatif, onglet Security)
Plan	Test sécurité (code)
Trace	<code>.github/workflows/codeql.yml</code>

Étant donné le code source JS/TS,

Quand CodeQL analyse le dépôt (push + hebdomadaire),

Alors les éventuelles vulnérabilités de code sont publiées dans Security > Code scanning pour traitement.

6. Scénarios couverts par l'automatisation (référence)

Ces comportements correspondent à une **suite automatisée** ; le scénario « métier » est : À chaque intégration, la chaîne bloque la livraison si la qualité n'est pas atteinte.

ID	Intention métier technique	Moyen
ST-AUTO-01	Pas de violation des règles ESLint	<code>npm run lint</code> dans la CI
ST-AUTO-02	Régression sur filtres ou calculs	<code>npm run test:run</code> (Vitest)
ST-AUTO-03	Régression login / santé exposition	<code>npm run test:e2e</code> (Playwright — <code>e2e/smoke.spec.ts</code>)
ST-AUTO-04	Artefact déployable	<code>npm run build</code> + vérif dossier dans CI
ST-AUTO-05	Risque dépendances prod maîtrisé	<code>npm run audit:prod</code> dans la CI

Détail d'exécution : [DEMO_EPREUVE.md](./1.2 Préparer le déploiement d'une application/1.2.8 Procédure d'exécution des tests (épreuve).md).

7. Gabarit pour nouveau scénario

Copier-colier et renseigner :

```
### ST-FXX — [Titre court]

| Champ | Détail |
|-----|-----|
| **Priorité** | Critique / Haute / Moyenne / Basse |
| **Type** | Manuel / Automatisé |
| **Plan** | [Référencer la ligne PLAN_TEST.md] |
| **Environnement** | DEV / STAGING / PROD |
| **Préconditions** | ... |

**Étant donné** ...

**Quand** ...
```

****Alors**** ...

****Trace**** | Exécutant : ____ – Date : ____ – Statut : Passé / Échec |

8. Suivi d'exécution

ID	Scénario	Exécutant	Date	Environnement	Résultat
ST-F01	Authentification	Playwright + revue PROD	2026-05-27	QA local + PROD	Passé
ST-F02	Chargement données	Playwright	2026-05-27	QA local (build:e2e)	Passé
ST-F03	Filtres	Playwright	2026-05-27	QA local	Passé
ST-F04	Responsive	Playwright	2026-05-27	QA local (375px)	Passé
ST-F05	Performance	Playwright	2026-05-27	QA local	Passé
ST-F06	Badge environnement	Playwright	2026-05-27	QA local + PROD	Passé
ST-SEC01	Headers	security:headers + Playwright	2026-05-27	PROD	Passé
ST-SEC02	Garde-fous applicatifs	security.test.ts	—	CI / local	Passé (15 tests)

Rapport détaillé : [RAPPORT_EXECUTION_TESTS.md](./1.2 Préparer le déploiement d'une application/1.2.9 Rapport d'exécution des tests.md)
Validation : [VALIDER_RESULTATS_TESTS.md](./1.2 Préparer le déploiement d'une application/1.2.6 Valider les résultats des tests.md) **Commande de reproduction :** npm run test:campaign

Note staging : l'URL `staging--dirmhublot.netlify.app` renvoie 404 (branch deploy à activer). Campagne validée en QA local + contrôles PROD en lecture seule.